Taller — JOINs, Funciones de Agregación y GROUP BY en PostgreSQL

Trabajo en clase

Universidad de Pamplona
Facultad de Ingenierías y Arquitectura
Programa de Ingeniería de Sistemas
Bases de Datos I — 2026-2
Prf. Juan Alejandro Carrillo Jaimes

Estudiante: Neyer Eueymar Chaves Hernandez
Código: 1005012186

Indicaciones

- El taller contiene **10 ejercicios prácticos** sobre los tres temas de la Unidad III: JOINs (`INNER`, `LEFT`, `RIGHT`, `NATURAL`, `FULL` y *self join*), funciones de agregación y las cláusulas `GROUP BY` y `HAVING`.
- Todos los ejercicios están basados en la base de datos `coffee_coop_db` de la cooperativa cafetera trabajada en clase, la misma del taller anterior.
- Se permite todo lo del taller anterior, más `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN` y `FULL JOIN` (siempre con `ON`), `NATURAL JOIN` (solo en el ejercicio 4), las funciones `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` (con `COUNT(DISTINCT ...)` y `ROUND(valor, 2)`), `GROUP BY` y `HAVING`.
- No se permiten subconsultas, `WITH`, vistas, funciones de ventana, `FILTER`, operadores de conjuntos, `CASE`, `COALESCE`, funciones de fecha, `CROSS JOIN`, `JOIN ... USING`, `DISTINCT` fuera de `COUNT`, `LIMIT`, `OFFSET` ni `FETCH`.
- Toda tabla se escribe con su esquema (`geo.`, `farm.`, `buy.`, `pay.`) y un alias `T1`, `T2`, `T3`... en el orden en que aparece. Todas las columnas se referencian con su alias de tabla.
- Las condiciones sobre columnas sin agregar van en el `WHERE`; el `HAVING` se reserva para condiciones sobre funciones de agregación.
- Los alias de columnas están en inglés y en formato *snake_case*.
- Cada literal se desarrolló en un archivo `ex_NN_x.sql` dentro de `content/coffee-coop/scripts/querys/02-sql-joins-group-by/`. Cuando la salida tiene muchas filas se muestran las primeras 10 y el total que reporta el cliente.

Tablas por esquema

Esquema	Tablas
`geo`	`departments`, `municipalities`
`farm`	`farmers`, `farms`, `plots`, `coffee_varieties`, `plot_varieties`, `certifications`, `farm_certifications`
`buy`	`collection_points`, `daily_prices`, `deliveries`, `quality_analyses`
`pay`	`settlements`, `payments`

Orden de ejecución lógico de una consulta

PostgreSQL no ejecuta la consulta en el orden en que se escribe. Primero arma las filas con el `FROM` y los `JOIN`, después filtra filas con el `WHERE`, luego forma los grupos y filtra grupos con el `HAVING`, y solo al final calcula el `SELECT` y ordena.

```
FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
une tablas    filtra   forma      filtra   columnas  ordena
              filas    grupos     grupos   y alias
```

- Una función de agregación no puede ir en el `WHERE`: los grupos todavía no existen. Va en el `HAVING`.
- Un alias definido en el `SELECT` sí puede usarse en el `ORDER BY`, pero no en el `WHERE` ni en el `HAVING`.
- Toda columna del `SELECT` que no esté dentro de una función de agregación debe aparecer en el `GROUP BY`.

Ejercicios

Parte A — JOINs

Ejercicio 1. INNER JOIN — Fincas, municipios y entregas

La coordinación regional necesita reportes que muestren nombres en lugar de códigos: el nombre del departamento en vez de su código DANE, el nombre del caficultor en vez de su identificador.

Tablas: `geo.municipalities`, `geo.departments`, `farm.farms`, `farm.farmers`, `buy.deliveries`, `buy.collection_points`

Literal a)

a) Consulte el código y el nombre de los municipios de Norte de Santander (código DANE 54) junto con el nombre del departamento, con los alias `municipality_code`, `municipality` y `department`, ordenados alfabéticamente por municipio.

Consulta — `ex_01_a.sql`

```
/*
Exercise 01 - Literal a
Municipalities of Norte de Santander with department name
*/
SELECT
    T1.municipality_code AS municipality_code,
    T1.municipality_name AS municipality,
    T2.department_name AS department
FROM geo.municipalities AS T1
INNER JOIN geo.departments AS T2
    ON T1.department_code = T2.department_code
WHERE T1.department_code = '54'
ORDER BY T1.municipality_name ASC;
```

Resultado obtenido en PostgreSQL

```
municipality_code | municipality | department
------------------+--------------+-------------------
54003             | Ábrego       | Norte De Santander
54051             | Arboledas    | Norte De Santander
54099             | Bochalema    | Norte De Santander
54109             | Bucarasica   | Norte De Santander
54128             | Cáchira      | Norte De Santander
54125             | Cácota       | Norte De Santander
54172             | Chinácota    | Norte De Santander
54174             | Chitagá      | Norte De Santander
54206             | Convención   | Norte De Santander
54001             | Cúcuta       | Norte De Santander
(10 rows)
```

Literal b)

b) Consulte el nombre de la finca, la vereda, el nombre del municipio y los nombres y apellidos del caficultor dueño de las fincas del municipio 54518, con los alias `farm`, `rural_district`, `municipality`, `first_name` y `last_name`, ordenadas por apellido del caficultor. (Tres tablas.)

Consulta — `ex_01_b.sql`

```
/*
Exercise 01 - Literal b
Farms, rural districts, municipality, and farmer full name for municipality 54518
*/
SELECT
    T1.farm_name AS farm,
    T1.rural_district AS rural_district,
    T2.municipality_name AS municipality,
    T3.first_name AS first_name,
    T3.last_name AS last_name
FROM farm.farms AS T1
INNER JOIN geo.municipalities AS T2
    ON T1.municipality_code = T2.municipality_code
INNER JOIN farm.farmers AS T3
    ON T1.farmer_id = T3.farmer_id
WHERE T1.municipality_code = '54518'
ORDER BY T3.last_name ASC;
```

Resultado obtenido en PostgreSQL

```
farm        | rural_district | municipality | first_name     | last_name
------------+----------------+--------------+----------------+------------------
El Jardín   | El Escorial    | Pamplona     | Manuel         | Agudelo Jiménez
La Selva    | Peñas          | Pamplona     | Manuel         | Agudelo Jiménez
El Diamante | El Alizal      | Pamplona     | Marleny        | Agudelo Sandoval
Bellavista  | Alto Grande    | Pamplona     | Rocío          | Álvarez Cárdenas
El Encanto  | Fontibón       | Pamplona     | Rocío          | Álvarez Cárdenas
Monteverde  | Lengua         | Pamplona     | Mónica         | Álvarez Carrillo
La Cabaña   | Lengua         | Pamplona     | Mónica         | Álvarez Carrillo
El Arrayán  | Navarro        | Pamplona     | Marleny        | Álvarez Correa
El Mango    | La Legua       | Pamplona     | Alberto        | Álvarez Villamizar
Monteverde  | La Legua       | Pamplona     | Myriam Yuliana | Arango Narváez
(10 rows)
```

Literal c)

c) Consulte el código de la entrega, el nombre de la finca, el nombre del punto de compra y el peso bruto de las entregas del 30 de septiembre de 2026 (use `LIKE` sobre `delivery_id`), con los alias `delivery`, `farm`, `collection_point` y `gross_weight_kg`, ordenadas por punto de compra y luego por peso de mayor a menor.

Consulta — `ex_01_c.sql`

```
/*
Exercise 01 - Literal c
Deliveries on September 30, 2026 with farm and collection point names
*/
SELECT
    T1.delivery_id AS delivery,
    T2.farm_name AS farm,
    T3.point_name AS collection_point,
    T1.gross_weight_kg AS gross_weight_kg
FROM buy.deliveries AS T1
INNER JOIN farm.farms AS T2
    ON T1.farm_id = T2.farm_id
INNER JOIN buy.collection_points AS T3
    ON T1.collection_point_id = T3.collection_point_id
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
ORDER BY T3.point_name ASC, T1.gross_weight_kg DESC;
```

Resultado obtenido en PostgreSQL

```
delivery            | farm          | collection_point              | gross_weight_kg
--------------------+---------------+-------------------------------+----------------
DLV-20260930-000164 | La Divisa     | Punto de Compra Abriaquí 1    | 48.18
DLV-20260930-000028 | El Refugio    | Punto de Compra Acevedo 1     | 258.00
DLV-20260930-000094 | El Refugio    | Punto de Compra Acevedo 1     | 246.96
DLV-20260930-000292 | La Florida    | Punto de Compra Acevedo 1     | 122.13
DLV-20260930-000202 | Los Guaduales | Punto de Compra Agrado 1      | 246.62
DLV-20260930-000080 | El Arrayán    | Punto de Compra Agrado 1      | 120.11
DLV-20260930-000075 | La Primavera  | Punto de Compra Alcalá 1      | 61.99
DLV-20260930-000137 | San Rafael    | Punto de Compra Aldana 1      | 209.02
DLV-20260930-000130 | El Refugio    | Punto de Compra Algeciras 1   | 290.55
DLV-20260930-000246 | Los Alpes     | Punto de Compra Algeciras 1   | 247.42
(10 rows)
```

Ejercicio 2. LEFT JOIN — Asociados sin finca y entregas sin análisis

La seccional de Pamplona sospecha que hay asociados inscritos que todavía no han registrado ninguna finca, y el laboratorio quiere saber qué entregas del día quedaron sin analizar.

Tablas: `farm.farmers`, `farm.farms`, `buy.deliveries`, `buy.quality_analyses`

Literal a)

a) Consulte el documento, nombres y apellidos de todos los asociados del municipio 54518 junto con el nombre de su finca, incluyendo a quienes no tienen finca registrada, con los alias `document`, `first_name`, `last_name` y `farm`.

Consulta — `ex_02_a.sql`

```sql
/*
Exercise 02 - Literal a
All farmers from municipality 54518 with their farm name (if any)
*/
SELECT
    T1.document_number AS document,
    T1.first_name AS first_name,
    T1.last_name AS last_name,
    T2.farm_name AS farm
FROM farm.farmers AS T1
LEFT JOIN farm.farms AS T2
    ON T1.farmer_id = T2.farmer_id
WHERE T1.municipality_code = '54518';
```

Resultado obtenido en PostgreSQL

```
document   | first_name      | last_name        | farm
-----------+-----------------+------------------+--------------
1061690671 | Guillermo Pedro | Arias Rodríguez  | La Selva
712758473  | Wilson          | López Villamizar | El Silencio
134410171  | Orlando         | Salazar Bermúdez | El Diamante
870240973  | Jairo           | Molina Pabón     | El Recuerdo
727945493  | Gloria          | Marín Rivera     | La Esperanza
549054771  | María           | Burbano Duarte   | El Nogal
549054771  | María           | Burbano Duarte   | La Divisa
622105662  | Alejandra       | Molina Torres    | La Carolina
622105662  | Alejandra       | Molina Torres    | La Cristalina
622105662  | Alejandra       | Molina Torres    | Villa Rosa
(10 rows)
```

Literal b)

b) A partir del literal a), muestre únicamente los asociados que no tienen ninguna finca registrada.

Consulta — `ex_02_b.sql`

```
/*
Exercise 02 - Literal b
Farmers from municipality 54518 with no registered farms
*/
SELECT
    T1.document_number AS document,
    T1.first_name AS first_name,
    T1.last_name AS last_name,
    T2.farm_name AS farm
FROM farm.farmers AS T1
LEFT JOIN farm.farms AS T2
    ON T1.farmer_id = T2.farmer_id
WHERE T1.municipality_code = '54518'
  AND T2.farm_id IS NULL;
```

Resultado obtenido en PostgreSQL

```
document | first_name | last_name | farm
---------+------------+-----------+------
(0 rows)
```

Literal c)

c) Consulte todas las entregas del 30 de septiembre de 2026 con el código del análisis de calidad y el puntaje en taza, incluyendo las entregas que aún no tienen análisis, con los alias `delivery`, `analysis` y `cup_score`, ordenadas por código de entrega.

Consulta — `ex_02_c.sql`

```
/*
Exercise 02 - Literal c
Deliveries on September 30, 2026 with quality analysis details
*/
SELECT
    T1.delivery_id AS delivery,
    T2.analysis_id AS analysis,
    T2.cup_score AS cup_score
FROM buy.deliveries AS T1
LEFT JOIN buy.quality_analyses AS T2
    ON T1.delivery_id = T2.delivery_id
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
ORDER BY T1.delivery_id ASC;
```

Resultado obtenido en PostgreSQL

```
delivery            | analysis           | cup_score
--------------------+--------------------+----------
DLV-20260930-000001 |                    |
DLV-20260930-000002 |                    |
DLV-20260930-000003 | QA-20260930-000001 |
DLV-20260930-000004 | QA-20260930-000002 |
DLV-20260930-000005 | QA-20260930-000026 | 78.56
DLV-20260930-000006 | QA-20260930-000018 | 79.98
DLV-20260930-000007 |                    |
DLV-20260930-000008 | QA-20260930-000007 |
DLV-20260930-000009 |                    |
DLV-20260930-000010 | QA-20260930-000006 |
(10 rows)
```

Ejercicio 3. RIGHT JOIN — Puntos de compra y variedades

El área de operaciones quiere ver la actividad de todos sus puntos de compra el 30 de septiembre de 2026, incluso de los que no recibieron café ese día. Asistencia técnica, por su parte, quiere saber qué variedades del catálogo no están sembradas en ningún lote.

Tablas: `buy.deliveries`, `buy.collection_points`, `farm.plot_varieties`, `farm.coffee_varieties`

Literal a)

a) Escribiendo `buy.deliveries` como T1 y `buy.collection_points` como T2, use `RIGHT JOIN` para listar el nombre de cada punto de compra con el código y el peso bruto de las entregas que recibió ese día, con los alias `collection_point`, `delivery` y `gross_weight_kg`. Los puntos sin entregas ese día deben aparecer. *Pista: piense si la condición de la fecha debe ir en el `ON` o en el `WHERE`, y compruébelo ejecutando las dos versiones.*

Consulta — `ex_03_a.sql`

```
/*
Exercise 03 - Literal a
Collection points activity on September 30, 2026 using RIGHT JOIN
*/
SELECT
    T2.point_name AS collection_point,
    T1.delivery_id AS delivery,
    T1.gross_weight_kg AS gross_weight_kg
FROM buy.deliveries AS T1
RIGHT JOIN buy.collection_points AS T2
    ON T1.collection_point_id = T2.collection_point_id
   AND T1.delivery_id LIKE 'DLV-20260930-%';
```

Resultado obtenido en PostgreSQL

```
collection_point                          | delivery            | gross_weight_kg
------------------------------------------+---------------------+----------------
Punto de Compra Jambaló 1                 | DLV-20260930-000001 | 51.53
Punto de Compra El Playón 1               | DLV-20260930-000083 | 89.95
Punto de Compra San Sebastián De Mariquita 1 | DLV-20260930-000084 | 131.42
Punto de Compra La Merced 1               | DLV-20260930-000085 | 239.20
Punto de Compra Nátaga 1                  | DLV-20260930-000086 | 84.05
Punto de Compra Algeciras 1               | DLV-20260930-000087 | 116.95
Punto de Compra Confines 1                | DLV-20260930-000088 | 220.61
Punto de Compra Villarrica 1              | DLV-20260930-000089 | 229.80
Punto de Compra Sevilla 1                 | DLV-20260930-000090 | 154.09
Punto de Compra Abriaquí 1                | DLV-20260930-000164 | 48.18
(10 rows)
```

Explicación — Qué pasa si la condición de fecha va en el WHERE

Cuando la fecha va en el `ON`, la base primero toma las entregas de ese día y luego las une con los puntos de compra. Si un punto no tuvo entregas, igual aparece en la lista, con la entrega vacía (`NULL`).

Si la fecha va en el `WHERE`, el filtro se aplica después de la unión. Los puntos sin entregas tienen la entrega vacía, y un valor vacío nunca cumple el `LIKE 'DLV-20260930-%'`. Entonces esas filas se eliminan.

El resultado es que el `RIGHT JOIN` se comporta como un `INNER JOIN` y desaparecen justo los puntos que queríamos ver: los que no recibieron café. Por eso la fecha debe ir en el `ON`.

Literal b)

b) A partir del literal a), muestre únicamente los puntos de compra que no recibieron entregas ese día.

Consulta — `ex_03_b.sql`

```
/*
Exercise 03 - Literal b
Collection points without deliveries on September 30, 2026
*/
SELECT
    T2.point_name AS collection_point,
    T1.delivery_id AS delivery,
    T1.gross_weight_kg AS gross_weight_kg
FROM buy.deliveries AS T1
RIGHT JOIN buy.collection_points AS T2
    ON T1.collection_point_id = T2.collection_point_id
   AND T1.delivery_id LIKE 'DLV-20260930-%'
WHERE T1.delivery_id IS NULL;
```

Resultado obtenido en PostgreSQL

```
collection_point             | delivery | gross_weight_kg
-----------------------------+----------+----------------
Punto de Compra Tesalia 3    |          |
Punto de Compra Marulanda 1  |          |
Punto de Compra Guática 2    |          |
Punto de Compra Chinácota 1   |          |
Punto de Compra El Tambo 2   |          |
Punto de Compra La Argentina 1|          |
Punto de Compra Hobo 1       |          |
Punto de Compra Caldono 1    |          |
Punto de Compra Yaguará 1    |          |
Punto de Compra Icononzo 1   |          |
(10 rows)
```

Literal c)

c) Escribiendo `farm.plot_varieties` como T1 y `farm.coffee_varieties` como T2, use `RIGHT JOIN` para listar las variedades que no están sembradas en ningún lote, con los alias `variety` y `species`.

Consulta — `ex_03_c.sql`

```
/*
Exercise 03 - Literal c
Coffee varieties not planted in any plot using RIGHT JOIN
*/
SELECT
    T2.variety_name AS variety,
    T2.species AS species
FROM farm.plot_varieties AS T1
RIGHT JOIN farm.coffee_varieties AS T2
    ON T1.variety_id = T2.variety_id
WHERE T1.plot_id IS NULL;
```

Resultado obtenido en PostgreSQL

```
variety | species
--------+---------
(0 rows)
```

Ejercicio 4. NATURAL JOIN — Unión automática

`NATURAL JOIN` une las tablas por todas las columnas que tengan el mismo nombre en ambas. Es cómodo, pero puede dar resultados inesperados si el modelo tiene columnas repetidas que no son la llave.

Tablas: `geo.municipalities`, `geo.departments`

Literal a)

a) Consulte el nombre del municipio y el nombre del departamento usando `NATURAL JOIN`, con los alias `municipality` y `department`, ordenados por departamento y luego por municipio.

Consulta — `ex_04_a.sql`

```
/*
Exercise 04 - Literal a
Municipalities and departments using NATURAL JOIN
*/
SELECT
    municipality_name AS municipality,
    department_name AS department
FROM geo.municipalities
NATURAL JOIN geo.departments
ORDER BY department_name ASC, municipality_name ASC;
```

Resultado obtenido en PostgreSQL

```
municipality | department
-------------+-----------
Leticia      | Amazonas
Puerto Nariño| Amazonas
Abejorral    | Antioquia
Abriaquí     | Antioquia
Alejandría   | Antioquia
Amagá        | Antioquia
Amalfi       | Antioquia
Andes        | Antioquia
Angelópolis  | Antioquia
Angostura    | Antioquia
(10 rows)
```

Literal b)

b) Escriba la misma consulta con `INNER JOIN ... ON`, indicando explícitamente la columna de unión.

Consulta — `ex_04_b.sql`

```
/*
Exercise 04 - Literal b
Municipalities and departments using explicit INNER JOIN ... ON
*/
SELECT
    T1.municipality_name AS municipality,
    T2.department_name AS department
FROM geo.municipalities AS T1
INNER JOIN geo.departments AS T2
    ON T1.department_code = T2.department_code
ORDER BY T2.department_name ASC, T1.municipality_name ASC;
```

Resultado obtenido en PostgreSQL

```
municipality | department
-------------+-----------
Leticia      | Amazonas
Puerto Nariño| Amazonas
Abejorral    | Antioquia
Abriaquí     | Antioquia
Alejandría   | Antioquia
Amagá        | Antioquia
Amalfi       | Antioquia
Andes        | Antioquia
Angelópolis  | Antioquia
Angostura    | Antioquia
(10 rows)
```

Literal c)

c) Compare el número de filas de a) y b). Explique con sus propias palabras por cuáles columnas unió el `NATURAL JOIN`, si los resultados coinciden y por qué en un proyecto real es preferible escribir el `ON`.

Explicación — Comparación entre el NATURAL JOIN y el INNER JOIN

El `NATURAL JOIN` une las tablas usando todas las columnas que se llaman igual en las dos. Entre municipios y departamentos, esa columna es `department_code`, que es la llave que los relaciona.

Los resultados de a) y b) coinciden: salen las mismas filas, en el mismo orden y con la misma cantidad. Esto pasa porque la única columna en común es la llave.

Aun así, en un proyecto real es mejor escribir el `ON`. El `NATURAL JOIN` decide solo por cuáles columnas unir. Si mañana alguien agrega una columna con el mismo nombre en las dos tablas (por ejemplo `created_at`), la unión cambia sin avisar y el resultado se daña.

Con el `ON` se ve claro qué columna se usa para unir, el código es más fácil de leer y revisar, y no se rompe cuando cambian las tablas.

Ejercicio 5. FULL JOIN — Auditoría de análisis y liquidaciones

Auditoría interna quiere comprobar que el proceso esté completo: toda entrega analizada del 30 de septiembre de 2026 debería tener su liquidación, y toda liquidación debería

tener un análisis previo.

Tablas: `buy.quality_analyses`, `pay.settlements`

Literal a)

a) Use `FULL JOIN` entre los análisis y las liquidaciones de las entregas de ese día (unidos por la entrega) para mostrar el código del análisis, si fue aceptado, el código de la liquidación y su estado, con los alias `analysis`, `accepted`, `settlement` y `settlement_status`.

Consulta — `ex_05_a.sql`

```
/*
Exercise 05 - Literal a
Audit between quality analyses and settlements for September 30, 2026 using FULL JOIN
*/
SELECT
    T1.analysis_id AS analysis,
    T1.is_accepted AS accepted,
    T2.settlement_id AS settlement,
    T2.status AS settlement_status
FROM buy.quality_analyses AS T1
FULL JOIN pay.settlements AS T2
    ON T1.delivery_id = T2.delivery_id
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
   OR T2.delivery_id LIKE 'DLV-20260930-%';
```

Resultado obtenido en PostgreSQL

```
analysis           | accepted | settlement          | settlement_status
-------------------+----------+---------------------+------------------
QA-20260930-000023 | true     |                     |
QA-20260930-000049 | true     |                     |
QA-20260930-000011 | true     |                     |
QA-20260930-000066 | true     | STL-20261001-000041 | cancelled
QA-20260930-000081 | true     |                     |
QA-20260930-000062 | true     | STL-20261001-000057 | pending
QA-20260930-000031 | true     |                     |
QA-20260930-000099 | true     | STL-20261001-000062 | partially_paid
QA-20260930-000072 | true     | STL-20261002-000041 | paid
QA-20260930-000007 | true     | STL-20261002-000005 | paid
(10 rows)
```

Literal b)

b) A partir del literal a), muestre únicamente las filas incompletas: análisis sin liquidación o liquidaciones sin análisis. Indique si las filas tienen sentido según el negocio (¿qué pasa con un análisis no aceptado?).

Consulta — `ex_05_b.sql`

```
/*
Exercise 05 - Literal b
Incomplete audit records: analyses without settlement or settlements without analysis
*/
SELECT
    T1.analysis_id AS analysis,
    T1.is_accepted AS accepted,
    T2.settlement_id AS settlement,
    T2.status AS settlement_status
FROM buy.quality_analyses AS T1
FULL JOIN pay.settlements AS T2
    ON T1.delivery_id = T2.delivery_id
WHERE (T1.delivery_id LIKE 'DLV-20260930-%' OR T2.delivery_id LIKE 'DLV-20260930-%')
  AND (T1.analysis_id IS NULL OR T2.settlement_id IS NULL);
```

Resultado obtenido en PostgreSQL

```
analysis           | accepted | settlement | settlement_status
-------------------+----------+------------+------------------
QA-20260930-000020 | true     |            |
QA-20260930-000054 | true     |            |
QA-20260930-000025 | false    |            |
QA-20260930-000095 | true     |            |
QA-20260930-000101 | true     |            |
QA-20260930-000090 | true     |            |
QA-20260930-000071 | true     |            |
QA-20260930-000135 | true     |            |
QA-20260930-000131 | true     |            |
QA-20260930-000030 | true     |            |
(10 rows)
```

Explicación — ¿Tienen sentido las filas del literal b)?

En su mayoría sí. Según el negocio, solo se liquidan las entregas aceptadas. Por eso un análisis no aceptado (`accepted = false`) sin liquidación es normal. Un ejemplo es `QA-20260930-000025`.

Las filas con análisis aceptado y sin liquidación son las que auditoría debe revisar. Ya deberían tener su liquidación, así que pueden estar atrasadas o pendientes de generar.

En las filas mostradas no aparece ninguna liquidación sin análisis. Eso es lo esperado, porque nunca se debería pagar una entrega que no fue analizada. Si apareciera una, sería un error grave del proceso.

Ejercicio 6. SELF JOIN — Vecinos de vereda

Para organizar rutas de recolección compartidas, la seccional de Pamplona quiere saber qué fincas están en la misma vereda, y para depurar el directorio quiere detectar asociados con apellidos repetidos.

Tablas: `farm.farms` (como T1 y T2), `farm.farmers` (como T1 y T2)

Literal a)

a) Consulte los pares de fincas del municipio 54518 que están en la misma vereda, con los alias `rural_district`, `farm_a` y `farm_b`, ordenados por vereda. Cada par debe aparecer

una sola vez y una finca no puede emparejarse consigo misma (use `T1.farm_id < T2.farm_id`).

Consulta — `ex_06_a.sql`

```
/*
Exercise 06 - Literal a
Pairs of farms located in the same rural district for municipality 54518
*/
SELECT
    T1.rural_district AS rural_district,
    T1.farm_name AS farm_a,
    T2.farm_name AS farm_b
FROM farm.farms AS T1
INNER JOIN farm.farms AS T2
    ON T1.rural_district = T2.rural_district
   AND T1.farm_id < T2.farm_id
WHERE T1.municipality_code = '54518'
ORDER BY T1.rural_district ASC;
```

Resultado obtenido en PostgreSQL

```
rural_district | farm_a      | farm_b
---------------+-------------+-----------
Alcaparral     | Miraflores  | Monte Alto
Alcaparral     | Miraflores  | La Granja
Alcaparral     | San José    | Monte Alto
Alcaparral     | San José    | La Granja
Alcaparral     | San José    | Los Pinos
Alcaparral     | San José    | Miraflores
Alcaparral     | La Floresta | Monte Alto
Alcaparral     | La Floresta | La Granja
Alcaparral     | La Floresta | Buenavista
Alcaparral     | La Floresta | San José
(10 rows)
```

Literal b)

b) Consulte los pares de asociados del municipio 54518 que tienen el mismo primer apellido, mostrando el apellido y los dos nombres completos, con los alias `last_name`, `farmer_a` y `farmer_b`. Cada par una sola vez.

Consulta — `ex_06_b.sql`

```
/*
Exercise 06 - Literal b
Pairs of farmers with the same first last name for municipality 54518
*/
SELECT
    T1.last_name AS last_name,
    T1.first_name AS farmer_a,
    T2.first_name AS farmer_b
FROM farm.farmers AS T1
INNER JOIN farm.farmers AS T2
    ON T1.last_name = T2.last_name
   AND T1.farmer_id < T2.farmer_id
WHERE T1.municipality_code = '54518';
```

Resultado obtenido en PostgreSQL

```
last_name        | farmer_a | farmer_b
-----------------+----------+---------------
López Villamizar | Wilson   | Fernando
López Villamizar | Wilson   | Laura
López Villamizar | Wilson   | Olga
López Villamizar | Wilson   | Daniela
López Villamizar | Wilson   | Esteban
López Villamizar | Wilson   | Rubén
López Villamizar | Wilson   | Rafael Ancízar
López Villamizar | Wilson   | Pedro
López Villamizar | Wilson   | Germán David
Molina Pabón     | Jairo    | Antonio José
(10 rows)
```

Parte B — Funciones de agregación

Ejercicio 7. COUNT, SUM, AVG, MIN y MAX — Resumen del día

La gerencia quiere un tablero con cifras de una sola línea sobre lo que pasó el 30 de septiembre de 2026. En este ejercicio no se usa `GROUP BY`: cada consulta devuelve una sola fila.

Tablas: `buy.deliveries`, `buy.quality_analyses`, `farm.farmers`

Literal a)

a) De las entregas de ese día, consulte el número total de entregas, la suma del peso bruto, el peso bruto promedio (redondeado a 2 decimales) y el número de fincas distintas que entregaron, con los alias `total_deliveries`, `total_gross_kg`, `avg_gross_kg` y `farms_delivering`.

Consulta — `ex_07_a.sql`

```
/*
Exercise 07 - Literal a
Summary of deliveries on September 30, 2026
*/
SELECT
    COUNT(T1.delivery_id) AS total_deliveries,
    SUM(T1.gross_weight_kg) AS total_gross_kg,
    ROUND(AVG(T1.gross_weight_kg), 2) AS avg_gross_kg,
    COUNT(DISTINCT T1.farm_id) AS farms_delivering
FROM buy.deliveries AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%';
```

Resultado obtenido en PostgreSQL

```
total_deliveries | total_gross_kg | avg_gross_kg | farms_delivering
-----------------+----------------+--------------+-----------------
300              | 52190.96       | 173.97       | 300
(1 row)
```

Literal b)

b) De los análisis de ese día, consulte el puntaje en taza mínimo, máximo y promedio, y la humedad mínima y máxima, con los alias `min_cup_score`, `max_cup_score`, `avg_cup_score`, `min_moisture_pct` y `max_moisture_pct`.

Consulta — `ex_07_b.sql`

```
/*
Exercise 07 - Literal b
Summary of quality metrics for analyses on September 30, 2026
*/
SELECT
    MIN(T1.cup_score) AS min_cup_score,
    MAX(T1.cup_score) AS max_cup_score,
    ROUND(AVG(T1.cup_score), 2) AS avg_cup_score,
    MIN(T1.moisture_pct) AS min_moisture_pct,
    MAX(T1.moisture_pct) AS max_moisture_pct
FROM buy.quality_analyses AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%';
```

Resultado obtenido en PostgreSQL

```
min_cup_score | max_cup_score | avg_cup_score | min_moisture_pct | max_moisture_pct
--------------+---------------+---------------+------------------+-----------------
77.17         | 87.28         | 82.91         | 8.39             | 17.68
(1 row)
```

Literal c)

c) De la tabla de asociados del municipio 54518, consulte el total de asociados (`COUNT(*)`) y el total de asociados con correo registrado (`COUNT` sobre la columna del correo), con los alias `total_farmers` y `farmers_with_email`. Explique por qué los dos conteos son distintos.

Consulta — `ex_07_c.sql`

```
/*
Exercise 07 - Literal c
Total farmers vs farmers with email in municipality 54518
*/
SELECT
    COUNT(*) AS total_farmers,
    COUNT(T1.email) AS farmers_with_email
FROM farm.farmers AS T1
WHERE T1.municipality_code = '54518';
```

Resultado obtenido en PostgreSQL

```
total_farmers | farmers_with_email
--------------+-------------------
230           | 165
(1 row)
```

Explicación — Por qué los dos conteos son distintos

`COUNT(*)` cuenta todas las filas, sin importar si tienen datos vacíos. Por eso da 230, que es el total de asociados de Pamplona.

`COUNT(T1.email)` solo cuenta las filas donde el correo tiene un valor. Ignora los correos vacíos (`NULL`). Por eso da 165.

La diferencia es de 65 asociados que no tienen correo registrado. El correo es un dato opcional, así que es normal que falte en algunos.

Parte C — GROUP BY y HAVING

Ejercicio 8. GROUP BY — Totales por grupo

Ahora las mismas cifras se necesitan desglosadas: por departamento, por tipo de café y por punto de compra.

Tablas: `geo.departments`, `geo.municipalities`, `buy.deliveries`, `buy.collection_points`

Literal a)

a) Consulte el nombre de cada departamento y su número de municipios, con los alias `department` y `municipalities_count`, ordenados de mayor a menor número de municipios.

Consulta — `ex_08_a.sql`

```
/*
Exercise 08 - Literal a
Number of municipalities per department
*/
SELECT
    T2.department_name AS department,
    COUNT(T1.municipality_code) AS municipalities_count
FROM geo.municipalities AS T1
INNER JOIN geo.departments AS T2
    ON T1.department_code = T2.department_code
GROUP BY T2.department_name
ORDER BY municipalities_count DESC;
```

Resultado obtenido en PostgreSQL

```
department         | municipalities_count
-------------------+---------------------
Antioquia          | 125
Boyacá             | 123
Cundinamarca       | 116
Santander          | 87
Nariño             | 64
Tolima             | 47
Bolívar            | 46
Valle Del Cauca    | 42
Cauca              | 42
Norte De Santander | 40
(10 rows)
```

Literal b)

b) De las entregas del 30 de septiembre de 2026, consulte por cada tipo de café el número de entregas y el peso bruto total, con los alias `coffee_type`, `deliveries_count` y `total_gross_kg`, ordenados por peso total de mayor a menor.

Consulta — `ex_08_b.sql`

```
/*
Exercise 08 - Literal b
Deliveries count and total weight by coffee type on September 30, 2026
*/
SELECT
    T1.coffee_type AS coffee_type,
    COUNT(T1.delivery_id) AS deliveries_count,
    SUM(T1.gross_weight_kg) AS total_gross_kg
FROM buy.deliveries AS T1
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
GROUP BY T1.coffee_type
ORDER BY total_gross_kg DESC;
```

Resultado obtenido en PostgreSQL

```
coffee_type   | deliveries_count | total_gross_kg
--------------+------------------+---------------
dry_parchment | 278              | 48246.55
cherry        | 10               | 2053.16
wet_parchment | 12               | 1891.25
(3 rows)
```

Literal c)

c) De las entregas de ese mismo día, consulte por cada punto de compra (mostrando su nombre) el número de entregas, el peso bruto total y el peso bruto promedio redondeado a 2 decimales, con los alias `collection_point`, `deliveries_count`, `total_gross_kg` y `avg_gross_kg`.

Consulta — `ex_08_c.sql`

```
/*
Exercise 08 - Literal c
Deliveries count, total weight, and average weight by collection point on September 30, 2026
*/
SELECT
    T2.point_name AS collection_point,
    COUNT(T1.delivery_id) AS deliveries_count,
    SUM(T1.gross_weight_kg) AS total_gross_kg,
    ROUND(AVG(T1.gross_weight_kg), 2) AS avg_gross_kg
FROM buy.deliveries AS T1
INNER JOIN buy.collection_points AS T2
    ON T1.collection_point_id = T2.collection_point_id
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
GROUP BY T2.point_name;
```

Resultado obtenido en PostgreSQL

```
collection_point            | deliveries_count | total_gross_kg | avg_gross_kg
----------------------------+------------------+----------------+-------------
Punto de Compra Abriaquí 1  | 1                | 48.18          | 48.18
Punto de Compra Acevedo 1   | 3                | 627.09         | 209.03
Punto de Compra Agrado 1    | 2                | 366.73         | 183.37
Punto de Compra Alcalá 1    | 1                | 61.99          | 61.99
Punto de Compra Aldana 1    | 1                | 209.02         | 209.02
Punto de Compra Algeciras 1 | 5                | 1093.81        | 218.76
Punto de Compra Altamira 1  | 2                | 284.88         | 142.44
Punto de Compra Andalucía 2 | 2                | 471.26         | 235.63
Punto de Compra Anserma 1   | 2                | 233.71         | 116.86
Punto de Compra Anserma 2   | 2                | 209.24         | 104.62
(10 rows)
```

Ejercicio 9. HAVING — Filtrar grupos

Los reportes del ejercicio anterior son largos; la gerencia solo quiere ver los grupos que superan ciertos umbrales.

Tablas: `geo.departments`, `geo.municipalities`, `buy.deliveries`, `buy.collection_points`, `farm.farms`, `buy.quality_analyses`

Literal a)

a) Repita el literal a) del ejercicio anterior mostrando solo los departamentos con más de 40 municipios.

Consulta — `ex_09_a.sql`

```sql
/*
Exercise 09 - Literal a
Departments with more than 40 municipalities
*/
SELECT
    T2.department_name AS department,
    COUNT(T1.municipality_code) AS municipalities_count
FROM geo.municipalities AS T1
INNER JOIN geo.departments AS T2
    ON T1.department_code = T2.department_code
GROUP BY T2.department_name
HAVING COUNT(T1.municipality_code) > 40
ORDER BY municipalities_count DESC;
```

Resultado obtenido en PostgreSQL

```
department      | municipalities_count
----------------+---------------------
Antioquia       | 125
Boyacá          | 123
Cundinamarca    | 116
Santander       | 87
Nariño          | 64
Tolima          | 47
Bolívar         | 46
Cauca           | 42
Valle Del Cauca | 42
(9 rows)
```

Literal b)

b) De las entregas del 30 de septiembre de 2026, consulte los puntos de compra (por nombre) que recibieron más de 1000 kg de peso bruto ese día, con los alias `collection_point` y `total_gross_kg`, ordenados de mayor a menor.

Consulta — `ex_09_b.sql`

```
/*
Exercise 09 - Literal b
Collection points with more than 1000 kg received on September 30, 2026
*/
SELECT
    T2.point_name AS collection_point,
    SUM(T1.gross_weight_kg) AS total_gross_kg
FROM buy.deliveries AS T1
INNER JOIN buy.collection_points AS T2
    ON T1.collection_point_id = T2.collection_point_id
WHERE T1.delivery_id LIKE 'DLV-20260930-%'
GROUP BY T2.point_name
HAVING SUM(T1.gross_weight_kg) > 1000
ORDER BY total_gross_kg DESC;
```

Resultado obtenido en PostgreSQL

```
collection_point                      | total_gross_kg
--------------------------------------+---------------
Punto de Compra Algeciras 1           | 1093.81
Punto de Compra Coromoro 1            | 1079.24
Punto de Compra San Andrés De Tumaco 1 | 1014.27
(3 rows)
```

Literal c)

c) Considerando todos los análisis registrados, consulte las fincas (por nombre) con al menos 3 análisis y un puntaje en taza promedio mayor o igual a 84, con los alias `farm`, `analyses_count` y `avg_cup_score` (redondeado a 2 decimales), ordenadas por promedio de mayor a menor. (Tres tablas.)

Consulta — `ex_09_c.sql`

```
/*
Exercise 09 - Literal c
Farms with at least 3 analyses and average cup score >= 84
*/
SELECT
    T3.farm_name AS farm,
    COUNT(T1.analysis_id) AS analyses_count,
    ROUND(AVG(T1.cup_score), 2) AS avg_cup_score
FROM buy.quality_analyses AS T1
INNER JOIN buy.deliveries AS T2
    ON T1.delivery_id = T2.delivery_id
INNER JOIN farm.farms AS T3
    ON T2.farm_id = T3.farm_id
GROUP BY T3.farm_name
HAVING COUNT(T1.analysis_id) >= 3
   AND AVG(T1.cup_score) >= 84
ORDER BY avg_cup_score DESC;
```

Resultado obtenido en PostgreSQL

```
farm            | analyses_count | avg_cup_score
```

Ejercicio 10. Integrador — JOIN + GROUP BY + HAVING

Tesorería y la seccional de Pamplona cierran el mes de septiembre de 2026 y necesitan reportes que combinan todo lo visto en la unidad. El código de entrega incluye la fecha: `DLV-YYYYMMDD-NNNNNN`.

Tablas: `pay.settlements`, `buy.deliveries`, `farm.farms`, `farm.farmers`, `geo.municipalities`

Literal a)

a) Consulte, por cada caficultor (documento, nombres y apellidos), el número de liquidaciones y el valor neto total liquidado por las entregas de septiembre de 2026, con los alias `document`, `first_name`, `last_name`, `settlements_count` y `total_net_cop`, ordenados por valor neto total de mayor a menor. (Cuatro tablas.)

Consulta — `ex_10_a.sql`

```
/*
Exercise 10 - Literal a
Settlement summary by farmer for September 2026
*/
SELECT
    T4.document_number AS document,
    T4.first_name AS first_name,
    T4.last_name AS last_name,
    COUNT(T1.settlement_id) AS settlements_count,
    SUM(T1.net_amount) AS total_net_cop
FROM pay.settlements AS T1
INNER JOIN buy.deliveries AS T2
    ON T1.delivery_id = T2.delivery_id
INNER JOIN farm.farms AS T3
    ON T2.farm_id = T3.farm_id
INNER JOIN farm.farmers AS T4
    ON T3.farmer_id = T4.farmer_id
WHERE T2.delivery_id LIKE 'DLV-202609%'
GROUP BY T4.document_number, T4.first_name, T4.last_name
ORDER BY total_net_cop DESC;
```

Resultado obtenido en PostgreSQL

```
document   | first_name      | last_name        | settlements_count | total_net_cop
-----------+-----------------+------------------+-------------------+---------------
30933664   | Cristian        | García Contreras |                 3 |   15493357.77
1071161211 | Alba            | Muñoz Ramírez    |                 2 |   14576631.94
526195285  | Jairo           | Valencia Zapata  |                 2 |   14528146.87
337551898  | Alejandro Mateo | Flórez Uribe     |                 2 |   14460246.30
854161034  | Luisa Martha    | Caicedo Romero   |                 1 |   13335230.83
782091682  | Luisa           | Duque Torres     |                 2 |   12843595.73
549378351  | Dora            | Galvis Ospina    |                 3 |   12821153.68
182907696  | Jorge           | Duarte Zapata    |                 2 |   12777306.01
446732017  | Reinaldo        | Ruiz Cuéllar     |                 2 |   12310076.05
628772920  | Aura            | Osorio Medina    |                 2 |   12133626.08
(10 rows)
```

Literal b)

b) A partir del literal a), muestre solo los caficultores con un valor neto total mayor a 5.000.000 COP.

Consulta — `ex_10_b.sql`

```
/*
Exercise 10 - Literal b
Farmers with total net settlement greater than 5,000,000 COP in September 2026
*/
SELECT
    T4.document_number AS document,
    T4.first_name AS first_name,
    T4.last_name AS last_name,
    COUNT(T1.settlement_id) AS settlements_count,
    SUM(T1.net_amount) AS total_net_cop
FROM pay.settlements AS T1
INNER JOIN buy.deliveries AS T2
    ON T1.delivery_id = T2.delivery_id
INNER JOIN farm.farms AS T3
    ON T2.farm_id = T3.farm_id
INNER JOIN farm.farmers AS T4
    ON T3.farmer_id = T4.farmer_id
WHERE T2.delivery_id LIKE 'DLV-202609%'
GROUP BY T4.document_number, T4.first_name, T4.last_name
HAVING SUM(T1.net_amount) > 5000000
ORDER BY total_net_cop DESC;
```

Resultado obtenido en PostgreSQL

```
document   | first_name      | last_name        | settlements_count | total_net_cop
-----------+-----------------+------------------+-------------------+--------------
30933664   | Cristian        | García Contreras | 3                 | 15493357.77
1071161211 | Alba            | Muñoz Ramírez    | 2                 | 14576631.94
526195285  | Jairo           | Valencia Zapata  | 2                 | 14528146.87
337551898  | Alejandro Mateo | Flórez Uribe     | 2                 | 14460246.30
854161034  | Luisa Martha    | Caicedo Romero   | 1                 | 13335230.83
782091682  | Luisa           | Duque Torres     | 2                 | 12843595.73
549378351  | Dora            | Galvis Ospina    | 3                 | 12821153.68
182907696  | Jorge           | Duarte Zapata    | 2                 | 12777306.01
446732017  | Reinaldo        | Ruiz Cuéllar     | 2                 | 12310076.05
628772920  | Aura            | Osorio Medina    | 2                 | 12133626.08
(10 rows)
```

Literal c)

c) Consulte, por cada municipio de Norte de Santander (mostrando su nombre), el número de fincas, el área total sumada y la altitud promedio redondeada a 2 decimales, con los alias `municipality`, `farms_count`, `total_area_ha` y `avg_altitude_masl`, mostrando solo los municipios con al menos 5 fincas y ordenados por altitud promedio de mayor a menor. Explique qué condición fue al `WHERE`, cuál al `HAVING` y por qué.

Consulta — `ex_10_c.sql`

```
/*
Exercise 10 - Literal c
Municipalities of Norte de Santander with at least 5 farms
*/
SELECT
    T1.municipality_name AS municipality,
    COUNT(T2.farm_id) AS farms_count,
    SUM(T2.total_area_ha) AS total_area_ha,
    ROUND(AVG(T2.latitude), 2) AS avg_altitude_masl
FROM geo.municipalities AS T1
INNER JOIN farm.farms AS T2
    ON T1.municipality_code = T2.municipality_code
WHERE T1.department_code = '54'
GROUP BY T1.municipality_name
HAVING COUNT(T2.farm_id) >= 5
ORDER BY avg_altitude_masl DESC;
```

Resultado obtenido en PostgreSQL

```
municipality | farms_count | total_area_ha | avg_altitude_masl
-------------+-------------+---------------+------------------
Sardinata    | 330         | 895.96        | 7.84
Cucutilla    | 315         | 806.32        | 7.84
Ocaña        | 339         | 836.98        | 7.83
Cácota       | 345         | 950.47        | 7.82
La Esperanza | 310         | 919.50        | 7.82
Villa Caro   | 372         | 947.97        | 7.82
Silos        | 329         | 929.93        | 7.82
Cáchira      | 311         | 828.40        | 7.82
Toledo       | 379         | 1040.38       | 7.81
Chitagá      | 334         | 893.56        | 7.81
(10 rows)
```

Explicación — Qué condición fue al WHERE, cuál al HAVING y por qué

Al `WHERE` fue `T1.department_code = '54'`. Es una condición sobre una columna normal, y se puede revisar fila por fila. Se aplica antes de agrupar, así que solo se agrupan los municipios de Norte de Santander.

Al `HAVING` fue `COUNT(T2.farm_id) >= 5`. Es una condición sobre una función de agregación: depende del número de fincas de cada municipio, y ese número solo existe después de formar los grupos.

El `WHERE` no puede usar funciones como `COUNT`, porque cuando se ejecuta los grupos todavía no existen. Por eso cada condición va en el lugar que le corresponde.